

OSSP Practical Experiments

Arun Tej(2520030472)

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
int i;

pid_t pid, status;


printf("Parent Process: PID = %d\n", getpid());


// Create 3 child processes
for (i = 1; i <= 3; i++) {
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child %d: PID = %d, PPID = %d\n",
            i, getpid(), getppid());

        sleep(i * 2);
    }
}
```

```
        printf("Child %d: PID = %d completed\n",
               i, getpid());

        exit(i);
    }
}

// Parent waits for any child
printf("\nParent using wait():\n");

for (i = 0; i < 2; i++) {
    pid = wait(&status);

    if (pid > 0) {
        printf("wait(): Child PID %d terminated\n",
               pid);
    }
}
```

```
// Parent waits for a specific child
printf("\nParent using waitpid():\n");

while ((pid = waitpid(-1, &status, 0)) > 0) {
    printf("waitpid(): Child PID %d
terminated\n", pid);
}

printf("\nAll child processes completed.\n");

return 0;
}
```

```
aruntej@LAPTOP-DGRT8C5G:~$ nano experi
aruntej@LAPTOP-DGRT8C5G:~$ gcc experim
aruntej@LAPTOP-DGRT8C5G:~$ ./experimen
Parent Process: PID = 659
Child 1: PID = 660, PPID = 659
Child 2: PID = 661, PPID = 659

Parent using wait():
Child 3: PID = 662, PPID = 659
Child 1: PID = 660 completed
wait(): Child PID 660 terminated
Child 2: PID = 661 completed
wait(): Child PID 661 terminated

Parent using waitpid():
Child 3: PID = 662 completed
waitpid(): Child PID 662 terminated

All child processes completed.
```

Zombie Process

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    pid_t pid = fork();
```

```
    if (pid < 0) {
```

```
        perror("fork failed");
```

```
        exit(1);
```

```
    }
```

```
    if (pid == 0) {
```

```
        printf("Child: PID = %d\n", getpid());
```

```
        printf("Child is terminating...\n");
```

```
        exit(0);
```

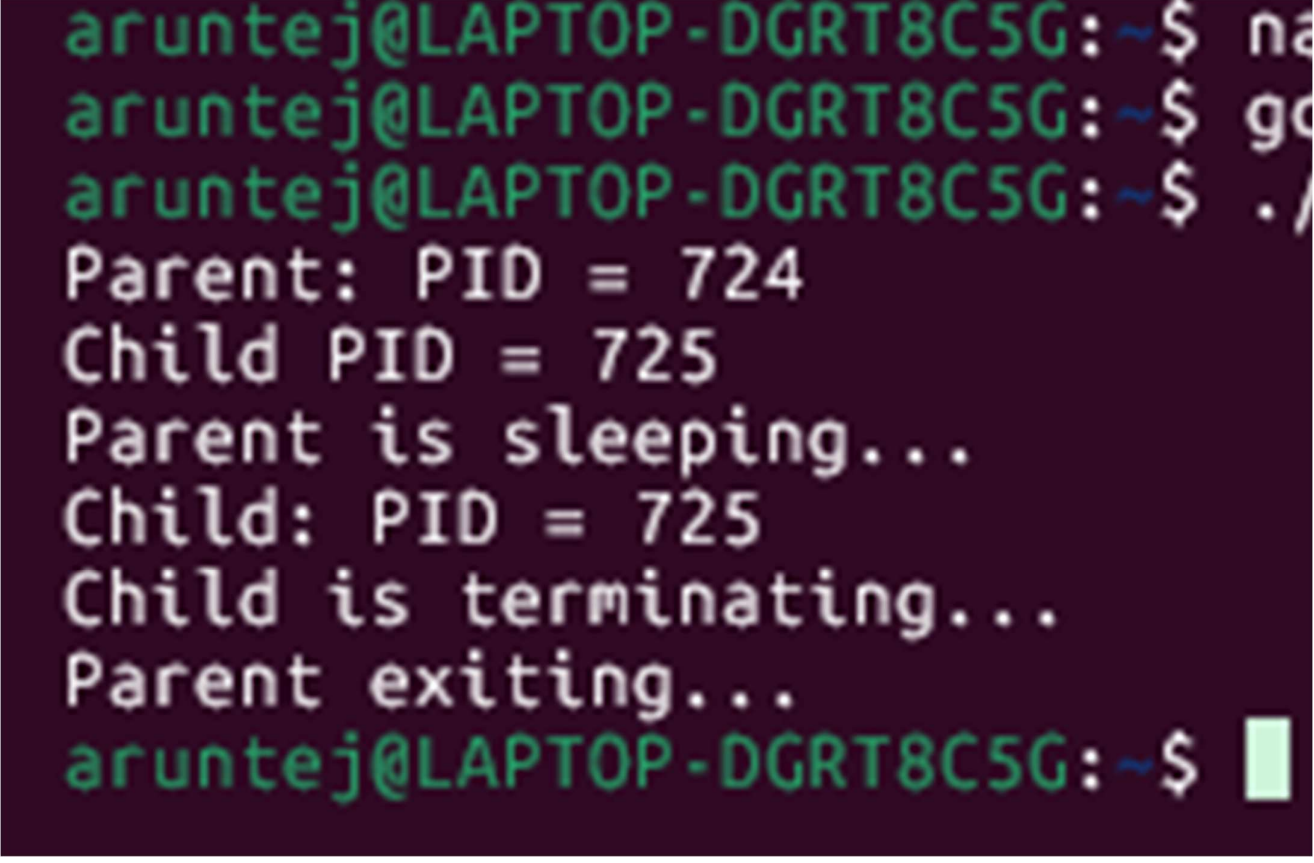
```
    }
```

```
    else {
```

```
        printf("Parent: PID = %d\n", getpid());
```

```
        printf("Child PID = %d\n", pid);
```

```
    printf("Parent is sleeping...\n");  
    sleep(30);  
  
    printf("Parent exiting...\n");  
}  
  
return 0;  
}
```

A terminal window with a dark purple background and green text. The prompt is 'aruntej@LAPTOP-DGRT8C5G: ~\$'. The user has entered 'na', 'go', and './'. The output shows the parent process printing its PID (724), the child process printing its PID (725), the parent sleeping, the child terminating, and the parent exiting. A green cursor is visible at the end of the last prompt line.

```
aruntej@LAPTOP-DGRT8C5G: ~$ na  
aruntej@LAPTOP-DGRT8C5G: ~$ go  
aruntej@LAPTOP-DGRT8C5G: ~$ ./  
Parent: PID = 724  
Child PID = 725  
Parent is sleeping...  
Child: PID = 725  
Child is terminating...  
Parent exiting...  
aruntej@LAPTOP-DGRT8C5G: ~$
```

